

# DETECCIÓN DE FRAUDES EN OFERTAS DE EMPLEO

Trabajo Final - Minería de Datos

Telmo Esparza

Aitzol Salaberria

Aitor Otegi

# Índice:

|                                                            |    |
|------------------------------------------------------------|----|
| 1. Introducción                                            | 3  |
| 2. Descripción del Dataset                                 | 4  |
| 2.1. Origen de los Datos                                   | 4  |
| 2.2. Variables de Entrada                                  | 4  |
| 2.3. Variable Objetivo                                     | 7  |
| 2.4. Relación entre las variables                          | 9  |
| 3. Métricas de Rendimiento                                 | 10 |
| 3.1. Elección de la Métrica                                | 10 |
| 3.2. Metodología de Validación                             | 10 |
| 4. Preparación de Datos                                    | 12 |
| 4.1. Pipeline y ColumnTransformer                          | 12 |
| 4.2. Limpieza de Formato                                   | 12 |
| 4.3. Imputación de Nulls                                   | 12 |
| 4.4. Codificación de variables                             | 13 |
| 4.5. Selección de variables                                | 14 |
| 5. Técnicas aplicadas para evitar desbalanceo              | 14 |
| 5.1. Muestreo de Datos                                     | 15 |
| 5.2. Ajuste del Umbral                                     | 15 |
| 6. Modelos Utilizados                                      | 16 |
| 6.1. KNN                                                   | 16 |
| 6.2. Árboles de Decisión                                   | 16 |
| 7. Experimentos Realizados                                 | 17 |
| 7.1. Modelo Base                                           | 17 |
| 7.2. Experimentos de imputación de valores nulos           | 18 |
| 7.2.1. Rellenado de salary-range 1-0                       | 18 |
| 7.2.2. Rellenado de salary-range discreto                  | 19 |
| 7.2.3. Imputación por moda                                 | 20 |
| 7.3.4. Imputación moda por grupos                          | 20 |
| 7.3. Experimentos en Numerización de Variables Categóricas | 21 |
| 7.3.1. Contar palabras/letras:                             | 22 |
| 7.3.2. Binary Encoder                                      | 23 |
| 7.3.3. Ordinal Encoder:                                    | 24 |
| 7.3.4. Count Vectorizer:                                   | 25 |
| 7.4. Experimentos Selección de Variables                   | 25 |
| 7.5. Experimentos Selección de Instancias                  | 27 |
| 7.6. Ajuste de Hiperparámetros y Búsqueda del mejor Umbral | 29 |
| 7.7 Experimentos con Árboles de Decisión:                  | 30 |
| 8. Modelo Definitivo                                       | 32 |
| 9. Interpretabilidad                                       | 33 |
| 9.1. Gráfico de barras                                     | 33 |
| 9.2. Gráfico de puntos                                     | 34 |
| 9.3. Gráfico de cascada                                    | 35 |
| 10. Conclusiones                                           | 36 |

# 1. Introducción

En este trabajo hemos utilizado un dataset de ofertas de empleo para resolver un problema de clasificación binaria no balanceada. El objetivo es predecir si una oferta es fraudulenta o real a partir de la información que contiene, como el título, la descripción, los requisitos, los beneficios, el tipo de contrato o la experiencia requerida.

Elegimos este problema porque tiene una aplicación bastante clara: en las plataformas de empleo pueden aparecer ofertas falsas, y sería útil poder detectar automáticamente cuáles tienen más probabilidad de ser fraudulentas.

La variable que queremos predecir es "fraudulent", que indica si una oferta es fraudulenta o no. Es un problema binario porque solo hay dos clases posibles: 0, oferta real, y 1, oferta fraudulenta.

Además, el problema está desbalanceado, ya que hay muchas más ofertas reales que fraudulentas. Por eso, no tiene mucho sentido fijarse solo en la accuracy, porque un modelo podría acertar muchas veces simplemente prediciendo siempre que la oferta es real. En este caso, nos interesa especialmente que el modelo sea capaz de detectar bien la clase minoritaria, es decir, las ofertas fraudulentas.

Por este motivo, en el trabajo se han usado métricas más adecuadas para este tipo de problemas, como la PR AUC, junto con otras métricas como precision, recall y F1-score. También se han probado distintas técnicas de preparación de datos y diferentes modelos, principalmente KNN y árboles de decisión, para analizar cómo afectan al resultado final.

## 2. Descripción del Dataset

### 2.1. Origen de los Datos

Para este trabajo hemos utilizado un dataset abierto llamado “Real or Fake Job Posting Prediction”, obtenido de Kaggle. Este conjunto de datos contiene información sobre ofertas de empleo publicadas online, algunas reales y otras fraudulentas.

Cada fila del dataset corresponde a una oferta de trabajo, y para cada una se recogen distintas características, como el título del puesto, la localización, el departamento, el perfil de la empresa, la descripción de la oferta, los requisitos, los beneficios, el tipo de contrato, la experiencia requerida, el sector y la función del puesto.

Este dataset resulta adecuado para el trabajo porque incluye variables de distintos tipos, tanto categóricas como textuales y numéricas/binarias, y además presenta un claro desbalanceo entre las clases.

### 2.2. Variables de Entrada

El dataset contiene distintas variables que describen cada oferta de empleo. Estas variables son de diferentes tipos: identificadoras, textuales, categóricas y binarias. La variable “fraudulent” no se considera variable de entrada, ya que es la variable objetivo que se quiere predecir.

| Variable | Significado                      | Tipo de dato             | Valores nulos |
|----------|----------------------------------|--------------------------|---------------|
| job_id   | Identificador único de la oferta | Numérico / identificador | 0%            |
| title    | Título de la oferta              | Texto                    | 0%            |
| location | Ubicación del                    | Texto                    | 0%            |

|                     |                                       |               |     |
|---------------------|---------------------------------------|---------------|-----|
|                     | puesto                                |               |     |
| department          | Departamento asociado al puesto       | Texto         | 65% |
| salary_range        | Rango salarial indicado en la oferta  | Texto / rango | 84% |
| company_profile     | Información sobre la empresa          | Texto largo   | 19% |
| description         | Descripción de la oferta              | Texto largo   | 0%  |
| requirements        | Requisitos del puesto                 | Texto largo   | 15% |
| benefits            | Beneficios ofrecidos                  | Texto largo   | 40% |
| telecommuting       | Indica si permite teletrabajo         | Binaria       | 0%  |
| has_company_logo    | Indica si la empresa tiene logo       | Binaria       | 0%  |
| has_questions       | Indica si la oferta incluye preguntas | Binaria       | 0%  |
| employment_type     | Tipo de empleo                        | Texto         | 19% |
| required_experience | Experiencia                           | Texto         | 39% |

|                    |                                 |       |     |
|--------------------|---------------------------------|-------|-----|
|                    | requerida                       |       |     |
| required_education | Formación requerida             | Texto | 45% |
| industry           | Sector o industria de la oferta | Texto | 27% |
| function           | Función o área del puesto       | Texto | 36% |

Además, las variables se pueden agrupar de la siguiente manera:

| Grupo de variables | Variables                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| Identificador      | job_id                                                                                                                  |
| Texto largo        | company_profile, description, requirements, benefits                                                                    |
| Texto              | title, location, department, salary_range, employment_type, required_experience, required_education, industry, function |
| Binarias           | telecommuting, has_company_logo, has_questions                                                                          |

En general, el conjunto de datos es interesante porque combina variables de distintos tipos. Algunas variables contienen texto libre, como la descripción o los requisitos de la oferta, mientras que otras son categóricas o binarias. Además, varias variables presentan valores perdidos, especialmente 'salary\_range', 'department', 'benefits', 'required\_education' y 'required\_experience', por lo que fue necesario aplicar técnicas de imputación antes de entrenar los modelos.

## 2.3. Variable Objetivo

La variable objetivo del dataset es "fraudulent", que indica si una oferta de empleo es real o fraudulenta. Esta variable es la que se quiere predecir mediante los modelos de clasificación.

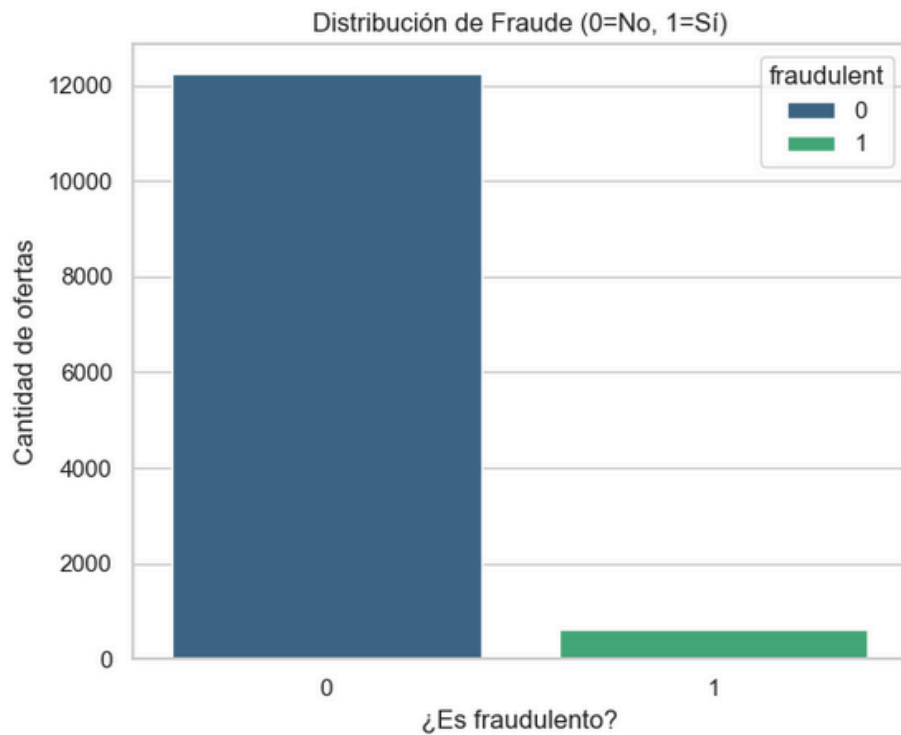
| Variable   | Significado                        | Tipo de dato | Valores nulos |
|------------|------------------------------------|--------------|---------------|
| fraudulent | Indica si la oferta es fraudulenta | Binaria      | 0%            |

Los valores posibles son:

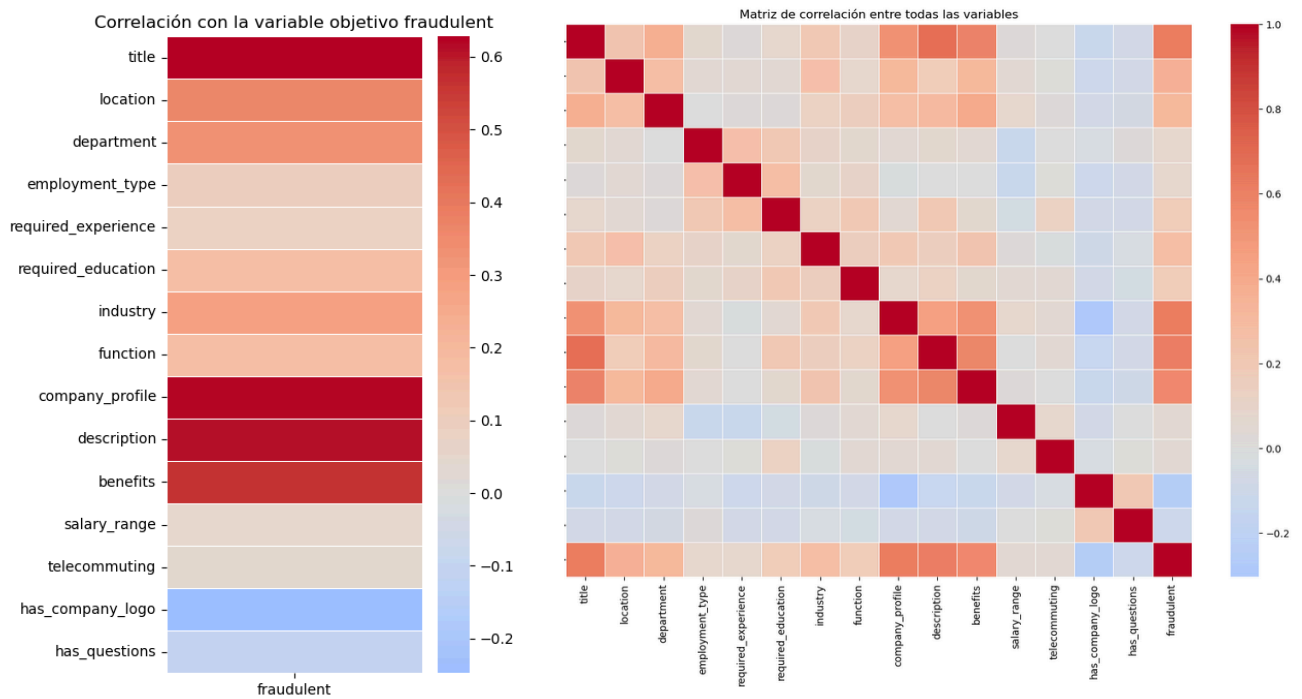
| Valor | Clase    | Significado           |
|-------|----------|-----------------------|
| 0     | Negativa | Oferta No Fraudulenta |
| 1     | Positiva | Oferta Fraudulenta    |

En este trabajo, la clase que más interesa detectar es la clase positiva, es decir, las ofertas fraudulentas. El problema está desbalanceado porque hay muchas más ofertas reales que fraudulentas, por lo que no sería suficiente evaluar los modelos únicamente con la accuracy.

Además, las ofertas fraudulentas representan menos del 5% del total del dataset, lo que confirma que se trata de un problema de clasificación no balanceada.



## 2.4. Relación entre las variables



En el gráfico de la izquierda podemos observar visualmente que correlación tienen las variables con la variable objetivo. Podemos ver que la mayoría de variables no tienen una correlación clara con la variable a predecir, aunque sí que hay algunas excepciones. 'title', 'company\_profile', 'description' y 'benefits' tienen una correlación positiva de más de un 0.5, lo nos será de utilidad más adelante. También es destacable que hay dos variables, 'has\_company\_logo' y 'has\_questions' que tienen una correlación negativa; aunque esto sí, inferior a las antes mencionadas, con una correlación aproximada de 0.1 y 0.2 respectivamente.

Por otro lado, en el gráfico de la derecha podemos ver las correlaciones de las variables entre ellas, aunque estos datos nos son bastante más superfluos, debido a que no se ve ninguna correlación demasiado significativa. Así que este gráfico, al contrario que el de la izquierda, no nos será de utilidad en el resto del proyecto.

## 3. Métricas de Rendimiento

### 3.1. Elección de la Métrica

Como el dataset está desbalanceado, no hemos usado "accuracy" como métrica principal. En este problema hay muchas más ofertas reales que fraudulentas, por lo que un modelo podría obtener una "accuracy" alta simplemente prediciendo casi siempre que las ofertas son reales.

Esto sería un problema, porque lo que más nos interesa es detectar correctamente las ofertas fraudulentas, que son la clase minoritaria. Por eso, hemos utilizado como métrica principal la "PR AUC", que mide el área bajo la curva Precision-Recall.

Esta métrica es más adecuada en problemas no balanceados porque se centra en el comportamiento del modelo sobre la clase positiva, en este caso las ofertas fraudulentas. Además, también hemos analizado otras métricas como "precision", "recall" y "F1-score", ya que nos ayudan a entender mejor el equilibrio entre detectar fraudes y no clasificar demasiadas ofertas reales como fraudulentas.

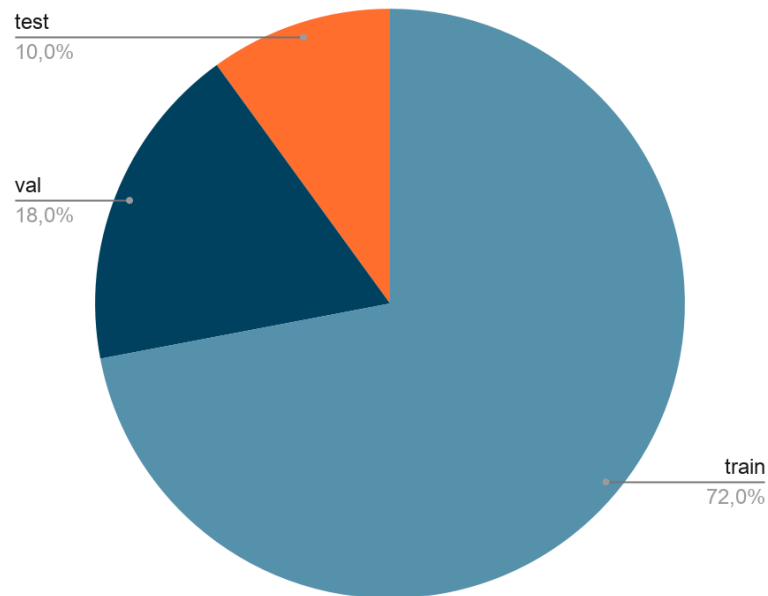
### 3.2. Metodología de Validación

Para evaluar los modelos, se dividió el dataset en tres conjuntos: entrenamiento, validación y test.

Primero se separó un 10% de los datos como conjunto de test. Este conjunto se reservó para evaluar el modelo final, por lo que no se utilizó durante el entrenamiento ni durante la elección de hiperparámetros.

Después, con el 90% restante, se hizo una segunda división para obtener los conjuntos de entrenamiento y validación. En esta segunda división, se reservó un 20% como conjunto de validación y el resto se utilizó como conjunto de entrenamiento.

Por tanto, la división final quedó aproximadamente así:



Todas las divisiones se hicieron de forma estratificada, usando “stratify”, para mantener aproximadamente la misma proporción de ofertas reales y fraudulentas en los tres conjuntos. Esto es importante porque el problema está desbalanceado y queremos que cada partición represente bien la distribución original de clases.

El conjunto de validación se utilizó para probar distintas configuraciones, como técnicas de preparación de datos, selección de variables, modelos e hiperparámetros. Una vez elegida la mejor configuración, el conjunto de test se utilizó únicamente para obtener una evaluación final más objetiva del modelo.

## 4. Preparación de Datos

### 4.1. Pipeline y ColumnTransformer

Como es lógico, para automatizar todo el proceso de transformación de los datos hemos hecho uso de las Pipelines y ColumnTransformer tanto de Sklearn como Imbalanced-Learn. Los ColumnTransformers nos permiten aplicar un preprocesamiento en concreto a columnas específicas fácilmente. Además combinado con las Pipelines de Sklearn o Imbalanced-Learn nos permite aplicar varios de estos preprocesamientos en serie. La diferencia entre las Pipelines de Sklearn e Imbalanced-Learn es que estas segundas permiten aplicar técnicas de muestreo de datos solo para los datos de Train.

### 4.2. Limpieza de Formato

El texto que nos suele venir de los dataset en crudo suele venir bastante “sucio”, por eso vamos a aplicar algunos preprocesamientos para intentar normalizarlos un poco,

1. **Limpiar espacios:** en caso de que algún campo tenga espacios de más en algún lugar este preprocesamiento lo elimina
2. **Minusculizar:** pasamos todos nuestros textos a minúsculas para minimizar posibles valores que probablemente signifiquen lo mismo
3. **Quitar símbolos:** quitamos símbolos como puntos, signos y demás para simplificar nuestros textos
4. **Quitar tildes:** al igual que con los símbolos, los eliminamos por simplificar los posibles valores

### 4.3. Imputación de Nulls

Uno de los primeros preprocesamientos que vamos a aplicar va a ser la imputación de nulos, esto debido a que hemos visto que tenemos variables con gran proporción de nulos.

Los diferentes métodos de imputación de nulos que vamos a probar serán:

1. **Imputación constante:** por la naturaleza de nuestro problema, que un valor no haya sido rellenado puede aportarnos información, por lo que en estas celdas incompletas vamos a añadir un valor constante como “unk” o -1 para diferenciarlo de los campos que sí han sido rellenados.

2. **Rellenado de Salary-Range 1-0:** como Salary-Range es una variable que tiene especialmente muchos valores nulos, vamos a preparar una transformación que simplemente guarde si esa celda a sido o no rellenada
3. **Rellenado de Salary-Range discreto:** similar al anterior, en vez de guardar solo si se ha rellenado o no vamos a guardar además en caso de ser rellenado si el rango del salario es alto ( $> 120000\$$  anuales), medio ( $< 120000\$$  y  $> 50000\$$  anuales) o bajo ( $< 50000\$$  anuales). Estos dos preprocesamientos pueden verse también como métodos de limpieza de formato.
4. **Imputación por moda:** incluimos una imputación por la moda, puesto que suele funcionar bastante bien en general
5. **Imputación por moda por grupos:** la imputación por moda puede generar relaciones artificiales entre las variables, para intentar minimizar este efecto vamos a aplicar la imputación por moda por grupos

## 4.4. Codificación de variables

Al estar usando KNN a la fuerza todas nuestras variables tienen que ser numéricas, pero nuestro dataset contiene variables categóricas prácticamente en su totalidad, por lo que este proceso es imprescindible en nuestro caso.

1. **Contar palabras/letras:** para nuestras variables de texto libre, como a priori tener una clase para cada una puede resultar en tener demasiados valores posibles, vamos a probar a contar su número de palabras y letras para discretizar las, a ver si la longitud de estos textos nos dice algo.
2. **Binary Encoder:** nuestros datos tienen muchos valores posibles, si usamos OneHotEncoding vamos a acabar generando infinidad de variables, la mayoría llenas de 0s, por tanto vamos a aplicar esta versión que genera muchas menos variables.
3. **Target Encoder:** este encoder cambia nuestros valores categóricos por la probabilidad que tienen de aparecer, si algún valor se repite mucho puede darnos información útil.
4. **Ordinal Encoder:** este transformador simplemente asigna un valor entero a cada valor posible, pero genera un orden falso. No le tenemos mucha expectativa a este encoder.
5. **Count Vectorizer:** Similar al Target Encoder, este divide los textos en tokens y cuenta la frecuencia de cada vector. En base a la frecuencia de los tokens de cada texto asigna un valor a cada campo.

## 4.5. Selección de variables

Como tenemos muchas variables, después de aplicar todo lo anterior vamos a probar un par de procesos de selección de variables para reducir la dimensionalidad de nuestro modelo y evitar variables que nos estén perjudicando.

Primeramente, vamos a comprobar si podemos eliminar alguna de las variables de texto libre. Para ello, al ser solo 4, vamos a probar toda la combinatoria de incluirlas o no y descubrir la mejor combinación.

Después, para limpiar el resto de variables, vamos a quedarnos con las  $k$  variables con más correlación del data. Para elegir  $k$  vamos a probar diferentes valores y quedarnos con aquella que de mejor rendimiento.

# 5. Técnicas aplicadas para evitar desbalanceo

## 5.1. Muestreo de Datos

Nuestro dataset tiene 12873 ejemplos, pero es bastante posible que no todos ellos nos ayuden a mejorar el rendimiento, pues algunos pueden ser atributos ruidosos. Además, al estar trabajando en un problema desbalanceado es muy posible que queramos generar más ejemplos de la clase minoritaria.

Para llevar a cabo estos objetivos haremos muestreo de datos, intentaremos tanto eliminar ejemplos de la clase negativa que nos puedan perturbar a la hora de predecir, cómo repetir algún ejemplo de la clase positiva para que en el KNN los ejemplos positivos tengan más ejemplos de su clase alrededor.

1. **CNN:** lo probaremos para eliminar aquellos ejemplos que no nos están aportando información al predecir train.
2. **Tomek Links:** un enlace de tomek son dos ejemplos que son el más cercano el uno del otro y son de clases diferentes. De estos enlaces vamos a probar a eliminar los ejemplos de la clase negativa.
3. **OSS:** consiste en aplicar el CNN y Tomek Links secuencialmente.
4. **NCL:** este sampler se centra en quitar ruido. Elimina ejemplos difíciles de la clase negativa y limpia las vecindades de ejemplos positivos difíciles.
5. **Random Over Sampler:** como tenemos muchos ejemplos de la clase positiva, este método duplica algunos ejemplos de esta clase para intentar balancear las clases.

Cabe resaltar que en ninguno de estos procesos eliminamos ejemplos de la clase minoritaria. Al tener muy pocos ejemplos de esta clase, vamos a suponer que todos son importantes.

## 5.2. Ajuste del Umbral

Tras probar y realizar todos los preprocesamientos pertinentes, probar varias combinaciones de hiperparámetros y encontrar la mejor solución en cuanto a superficie bajo la curva PR vamos a ajustar todavía más nuestros modelos y haremos una búsqueda del mejor umbral.

Para ello no nos queda otra más que probar todos las posibles probabilidades que nos de el cálculo de la curva de PR.

## 6. Modelos Utilizados

### 6.1. KNN

Como modelo principal para el trabajo hemos usado el algoritmo KNN, que clasifica un dato nuevo basándose en su similitud con los datos vecinos. Hemos utilizado KNN en todas las pruebas enfocadas en la toma de decisiones del modelo final, incluyendo la definición de los pipelines y el ajuste de hiperparámetros.

### 6.2. Árboles de Decisión

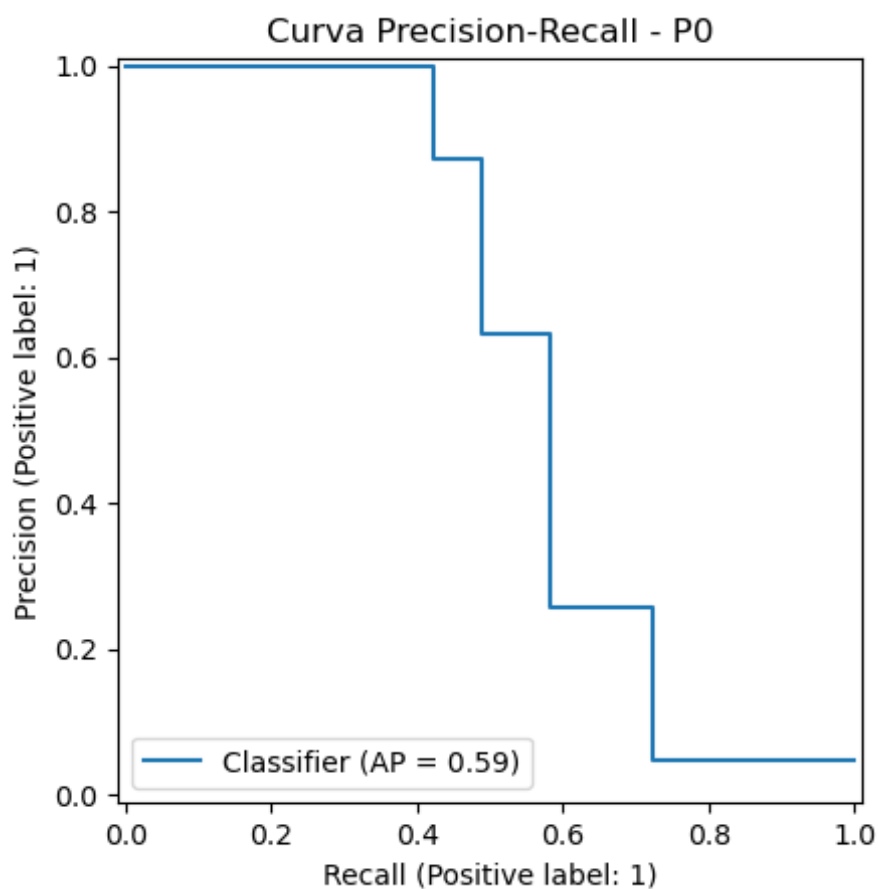
Hemos implementado también un árbol de decisión, tal como nos pedía la práctica. Sin embargo, para este modelo hemos usado los mismos pipelines que ajustamos con KNN. Sabemos que esto no es lo más óptimo, pero, dado el tiempo que teníamos para hacer el proyecto, nos ha parecido lo más razonable.

# 7. Experimentos Realizados

## 7.1. Modelo Base

Como primera prueba de todas, vamos a plantear un modelo básico funcional, para poder tener una primera aproximación de los resultados e iterar y probar combinaciones a partir de aquí.

Esta pipeline comenzará aplicando todos los métodos de limpieza de formato y tratamiento de texto que hemos comentado anteriormente, imputación de nulls con valor constante y codificador target encoder.



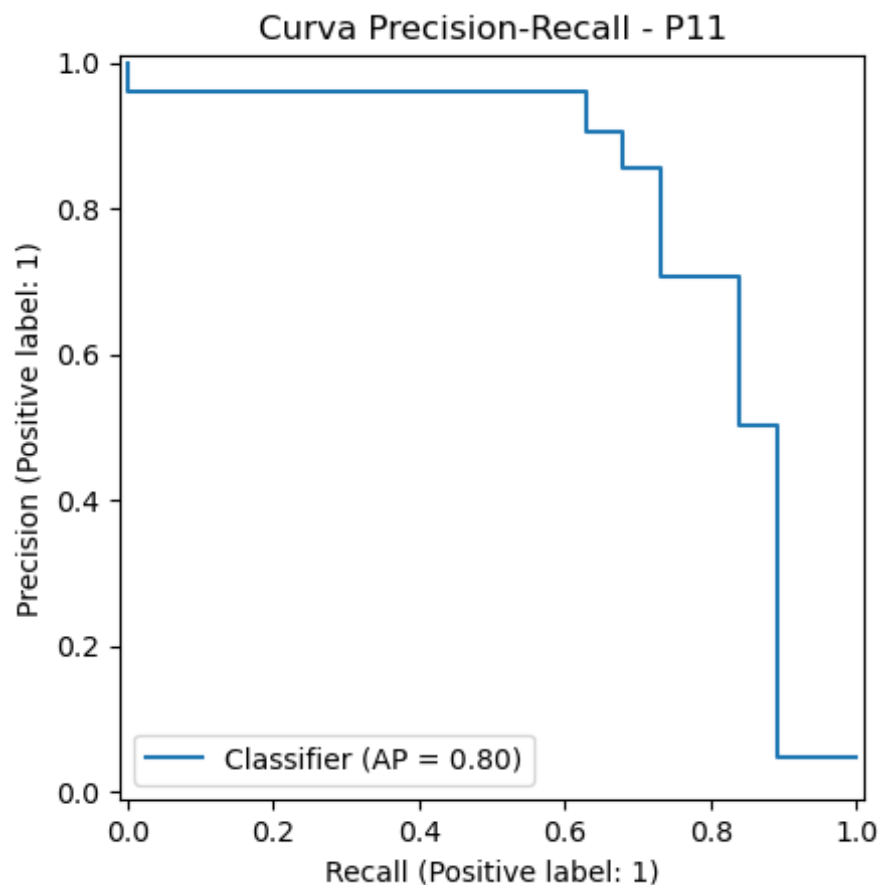
Como vemos esta primera aproximación resulta bastante mediocre, pero conforme vayamos realizando más pruebas estos resultados irán mejorando.

## 7.2. Experimentos de imputación de valores nulos

Para esta primera prueba “de verdad” vamos a probar los diferentes métodos de imputar valores nulos que hemos enumerado anteriormente.

### 7.2.1. Rellenado de salary-range 1-0

Para esta prueba solo vamos a cambiar la imputación de nulos en la variable del rango del

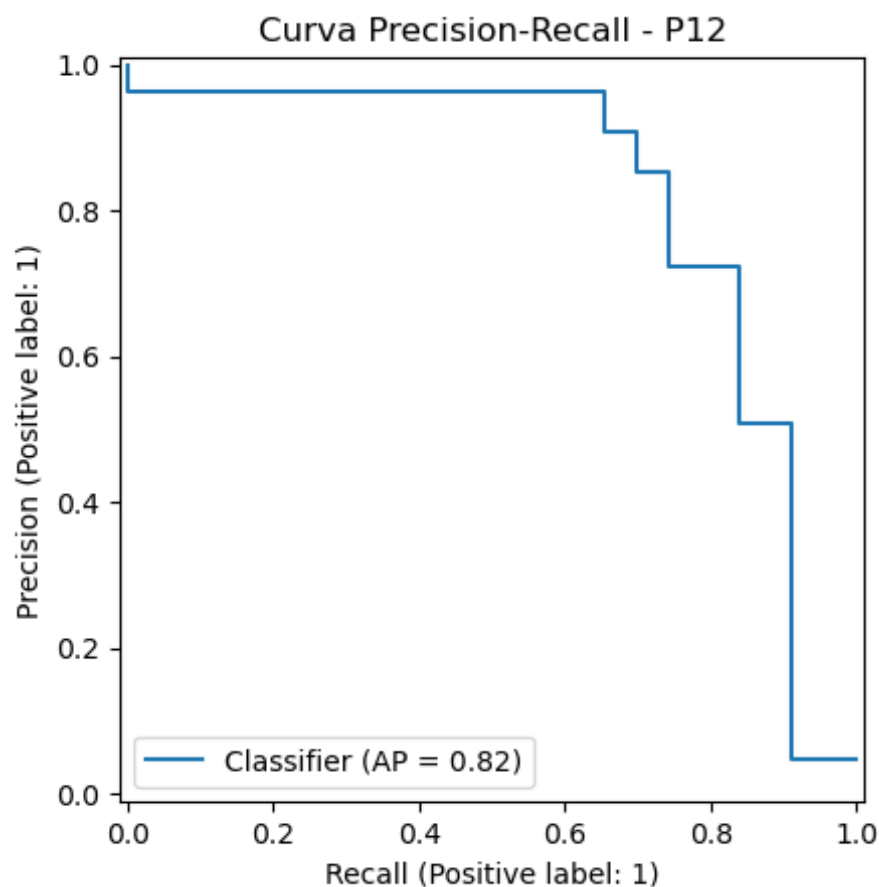


salario, el resto lo mantendremos igual:

Inmediatamente observamos que esto nos aporta una mejora muy sustancial, lo que puede ser síntoma de que realmente no importa en salario concreto sino solamente si el salario ha sido o no rellenado.

## 7.2.2. Rellenado de salary-range discreto

Vamos a probar la versión un poco más refinada del método anterior, a ver si notamos

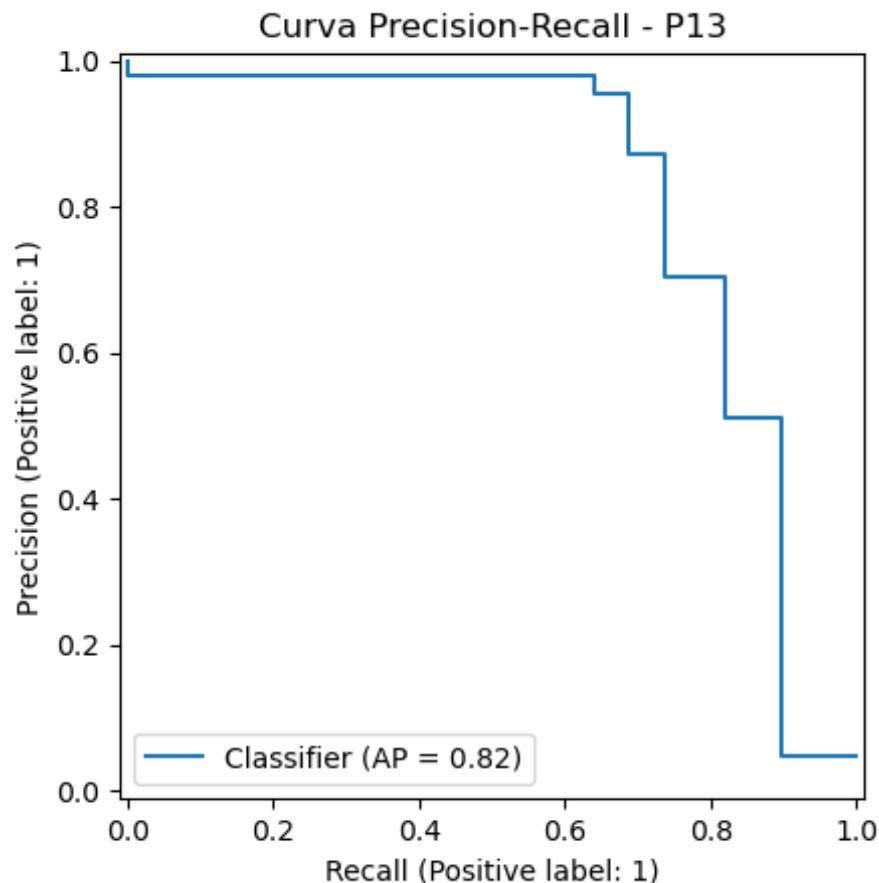


alguna diferencia.

Aunque no sea demasiado sustancial respecto a la anterior, si notamos una pequeña mejoría. Aunque diferenciar el nivel de salario que ofrecen las ofertas nos ha permitido refinar un poquito vemos que sin duda lo que más ha hecho es distinguir si el campo estaba en un primer momento rellenado o no. Debido a este éxito, seguiremos utilizando este preprocesamiento en las siguientes pruebas.

### 7.2.3. Imputación por moda

Ahora, con el resto de variables tanto categóricas como numéricas vamos a intentar imputar valores con la moda de cada variable.



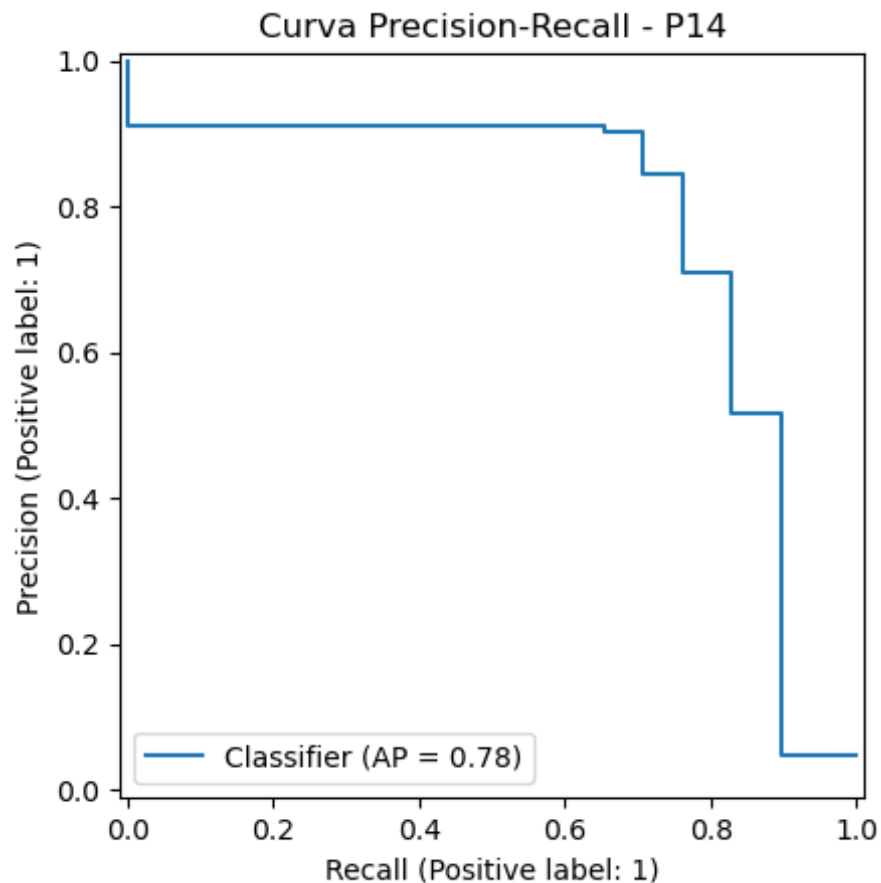
Observamos que la diferencia es prácticamente inapreciable, pero es posible que estemos generando alguna relación artificial que antes no existía.

### 7.3.4. Imputación moda por grupos

Para evitar el problema que comentábamos en el punto anterior, vamos a agrupar los datos antes de imputar por moda en algunas variables.

Las agrupaciones serán, teniendo en cuenta que la primera es la variable por la que vamos a agrupar y la segunda en la que vamos a imputar, department -> employment type, department -> industry, employment\_type -> required\_experience y required\_experience -> required\_education.

Hemos aplicado estas agrupaciones porque consideramos que se podrían intuir unas a partir de las otras.



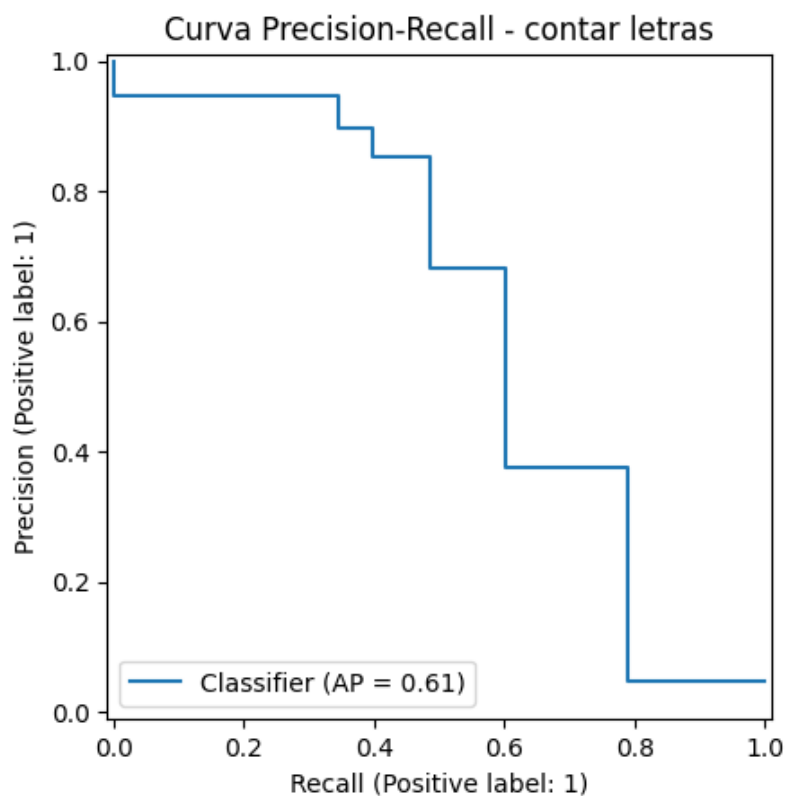
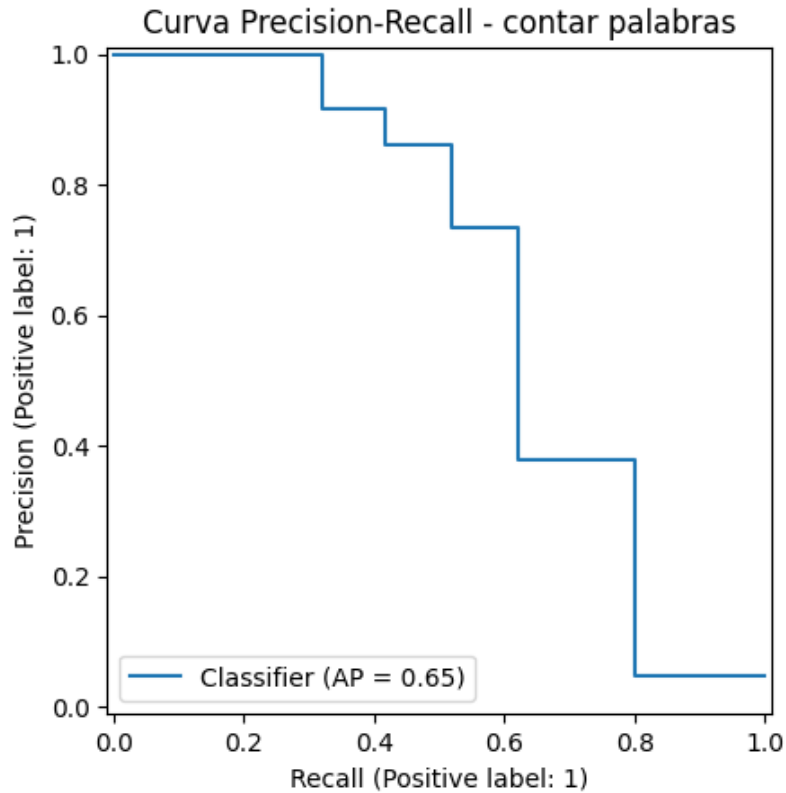
Como vemos que estas últimas pruebas no han tenido éxito y la imputación por moda simple nos aportaba una mejora inapreciable, vamos a mantener la imputación constante, por simplicidad del proceso y acortar tiempo de ejecución en un futuro.

## 7.3. Experimentos en Numerización de Variables Categóricas

Ahora experimentamos con diferentes pruebas de pasar variables categorías a numéricas.

### 7.3.1. Contar palabras/letras:

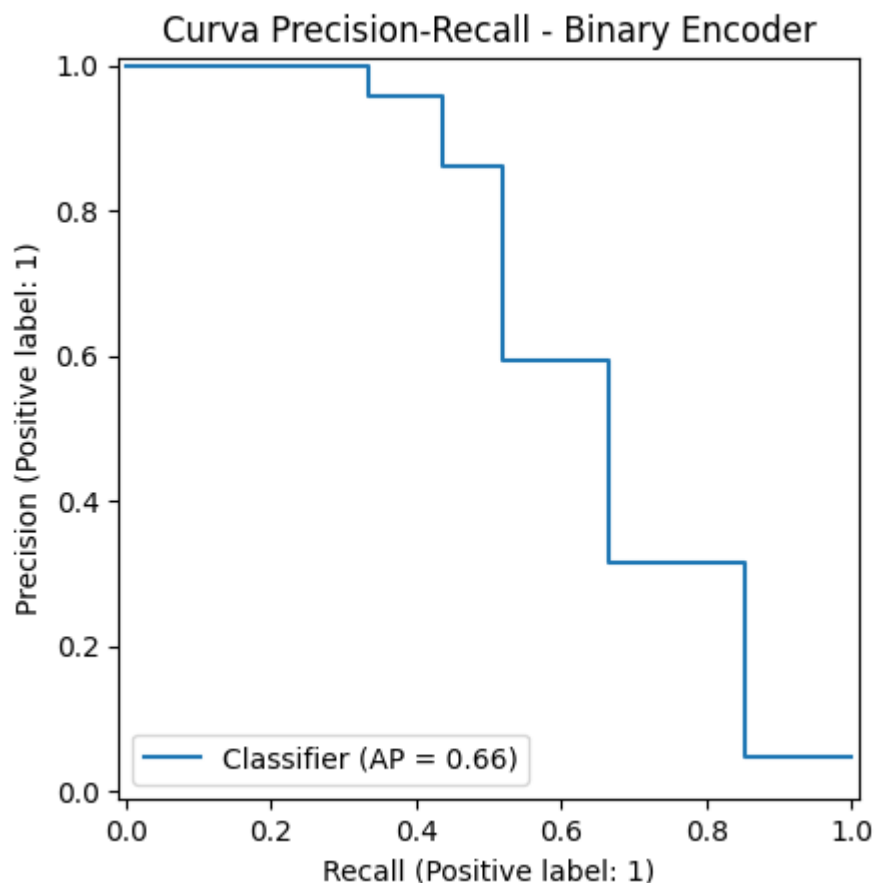
En esta prueba, solo para las variables de texto libre, vamos a probar a pasarlas a aplicarles el transformador de contar palabras y letras, posterior a eso aplicaremos un StandardScaler.



Observamos que ambos métodos han resultado en un fracaso, y resulta mejor trabajar con ambas variables directamente con el Target Encoder. Tras meditarlo pensamos que puede ser porque existan textos que se repiten en muchas ofertas de trabajo fraudulentas, entonces transformarlas de esta manera hace que perdamos información.

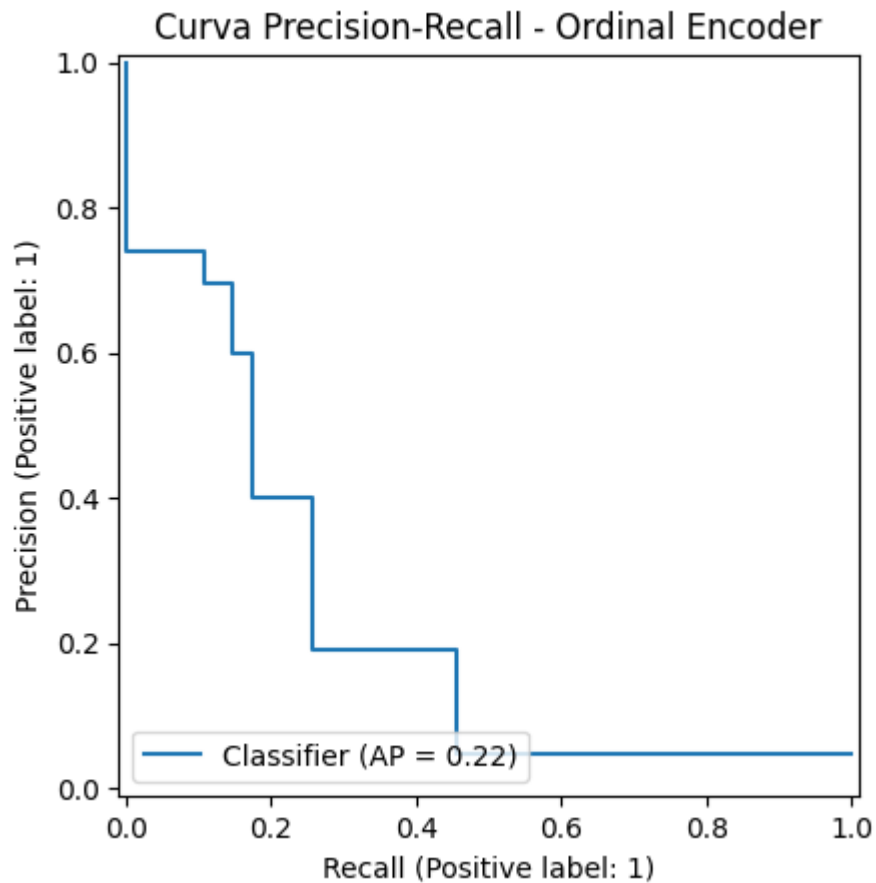
## 7.3.2. Binary Encoder

A partir de ahora las pruebas que realicemos las haremos sobre todas las variables, menos las binarias y salary\_range.



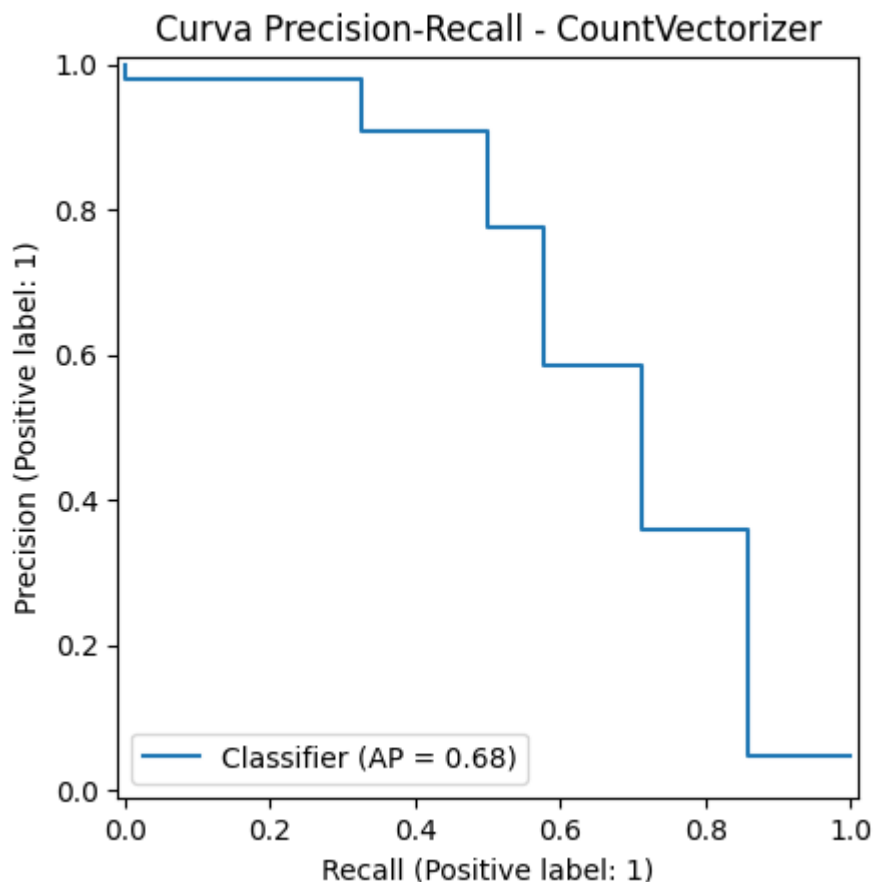
Una vez más, no parece que nos aporte mucha mejoría este método.

### 7.3.3. Ordinal Encoder:



Como ya vaticinamos anteriormente, este método resulta el peor de todos, puesto que estamos ordenando los valores posibles, cuando no existe ningún orden en casi todas las variables.

### 7.3.4. Count Vectorizer:



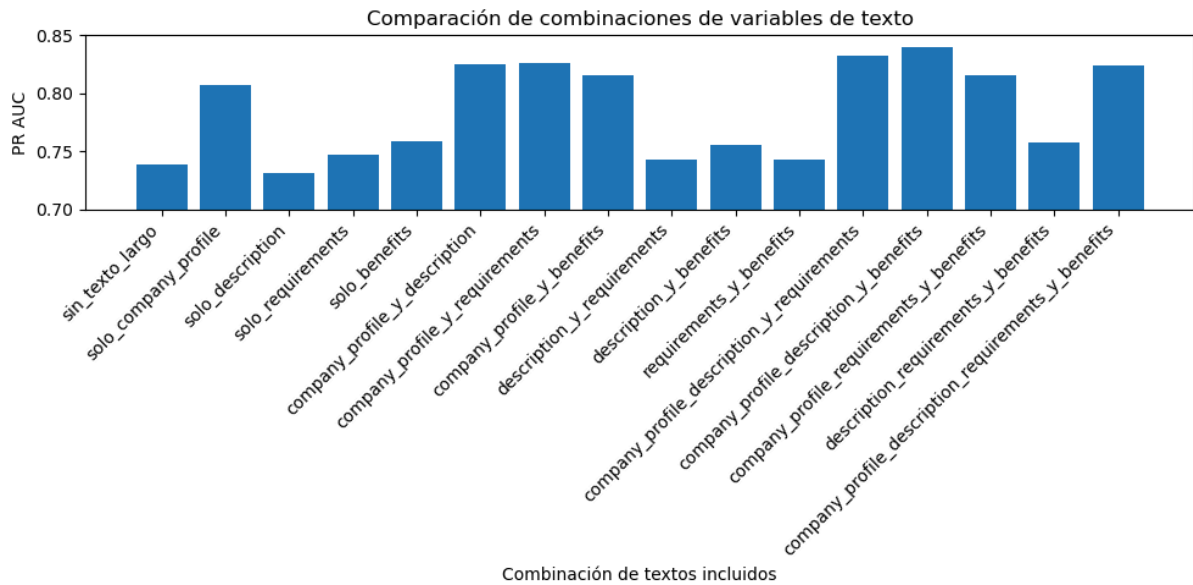
Y con esta prueba finalizamos todas las pruebas es fácil deducir que no deberíamos haber modificado nuestra forma de numerar variables categóricas, por lo que mantendremos el Target Encoder.

## 7.4. Experimentos Selección de Variables

Hasta ahora hemos trabajado con todas las variables, pero es posible que no todas nos aporten información o incluso que haga alguna que nos haga fallar, por eso vamos a intentar reducir nuestro abanico de variables para quedarnos solo con las que nos aporten información

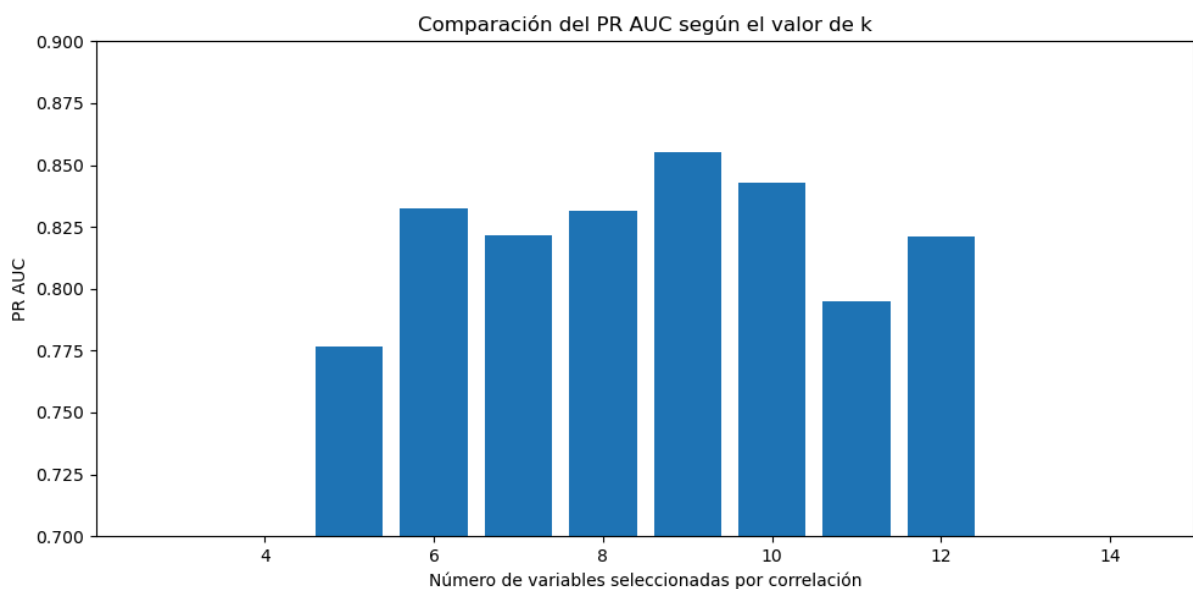
Primeramente vamos a estudiar si podemos eliminar alguna de las variables de texto largo, y si debemos mantener alguna, cuales.

Como solo tenemos 4 variables de texto, vamos a probar toda la combinatoria de introducir o no las variables, y quedarnos con el mejor resultado:



Aunque en esta gráfica cuesta un poco apreciarlo, si vemos la tabla de los rendimientos obtenidos con cada combinación resulta que lo mejor es deshacernos de la columna requirements y mantener las demás, por lo que en nuestras siguientes pruebas nunca más incluiremos esta variable.

Por último, para reducir todo lo posible la dimensionalidad de los datos que usamos para entrenar, como ya hemos comentado previamente, vamos usar un filtro para quedarnos con los atributos con mayor correlación con la variable de salida.

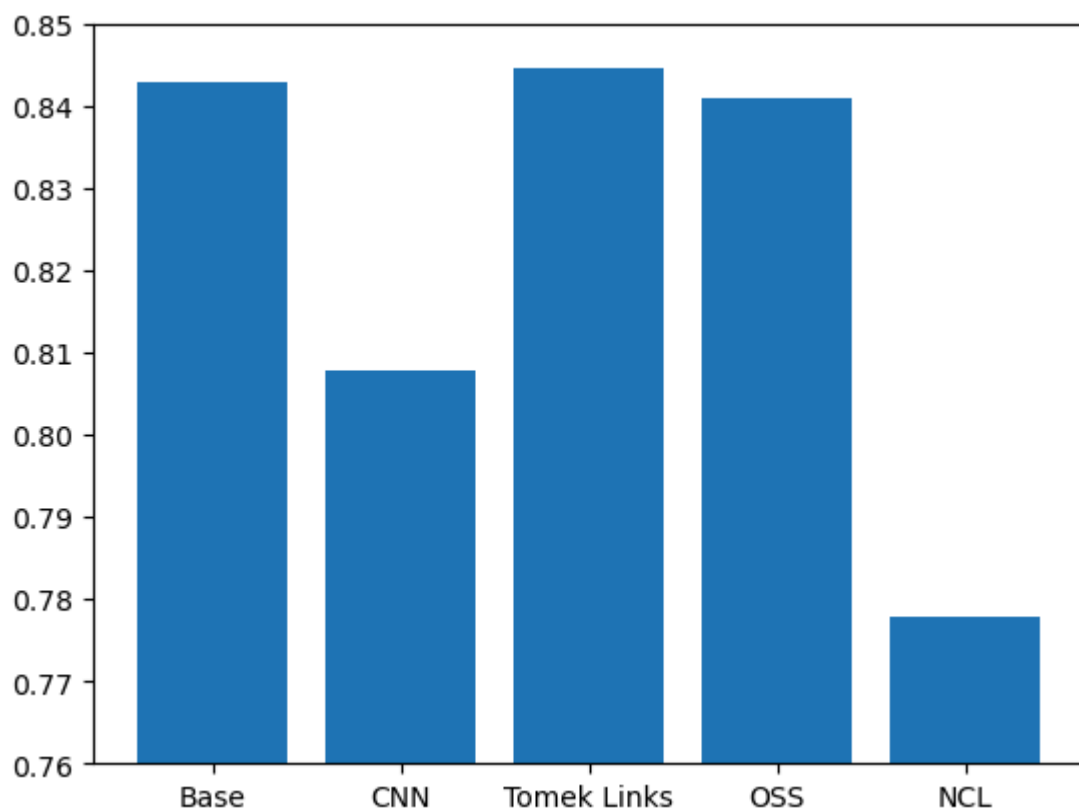


Tras las pruebas obtenemos que con las 9 variables con mayor correlación obtenemos el rendimiento máximo. Dados estos resultados, de aquí en adelante aplicaremos siempre este filtro, fijando la k a 9.

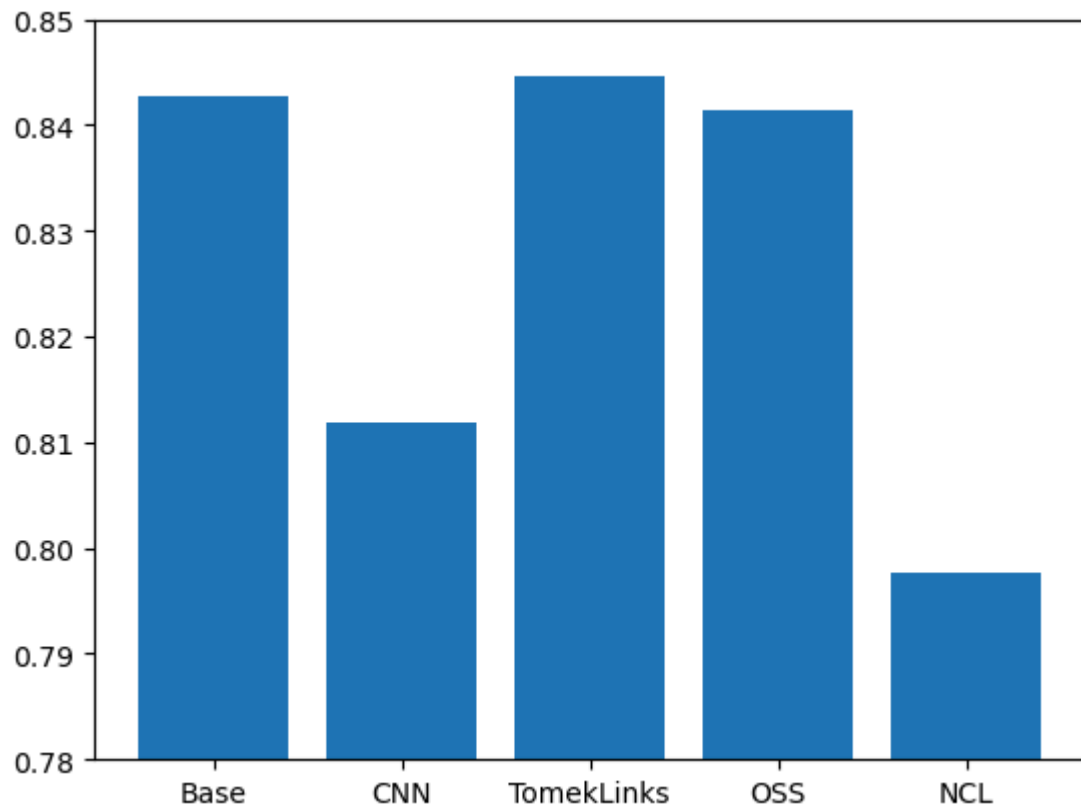
## 7.5. Experimentos Selección de Instancias

Al igual que nos pasaba con las variables, es muy posible que algunos ejemplos estén envenenando el dataset y haciendo que no tengamos nuestro máximo rendimiento. Además, al tener un dataset tan desbalanceado, nos podría interesar añadir más ejemplos de la clase positiva para balancearlos. En esto vamos a trabajar durante este experimento.

Primero vamos a probar a eliminar ejemplos ruidosos de la clase negativa, para dejar solo aquellos que nos ayuden a predecir mejor. Para ello vamos a probar todos los métodos de detección de ruido.

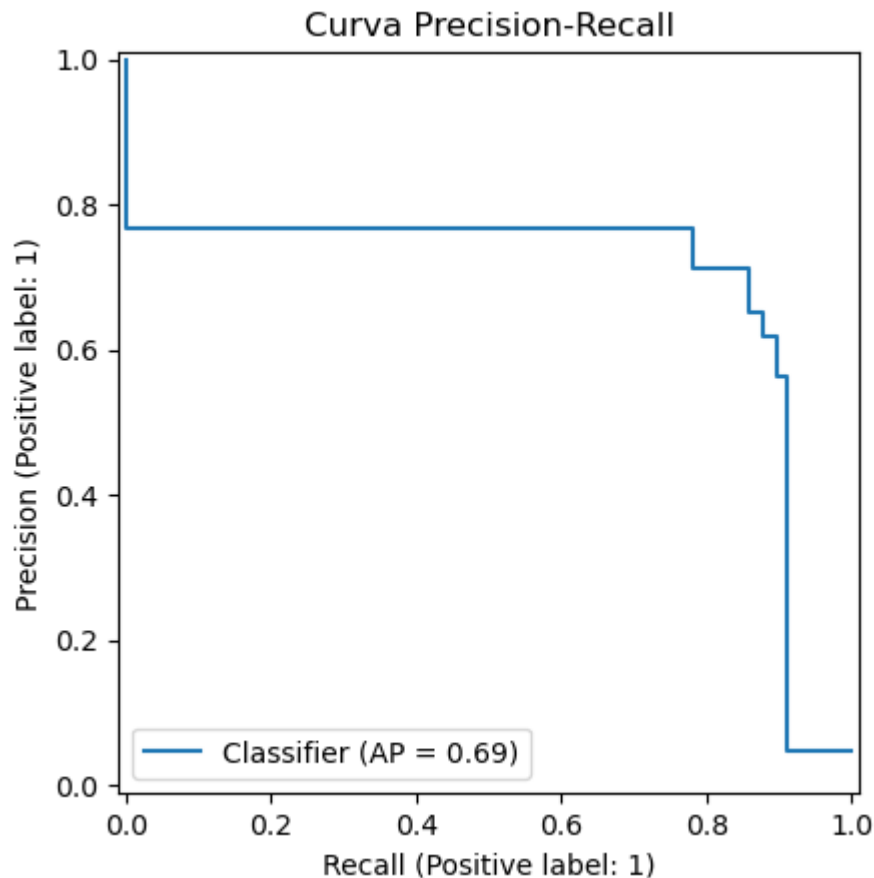


Tras probar todos los métodos, obtenemos que el único método que realmente mejora los resultados es usando Tomek Links. Por otro lado, es posible que si probáramos otros parámetros en los otros métodos estos funcionen mejor.



Tras comparar los resultados otra vez con los mejores parámetros para todas las funciones obtenemos resultados prácticamente idénticos. Por tanto vamos a empezar a aplicar Tomek Links para eliminar ruido en la clase negativa.

Pero antes de eso vamos a probar el último método que nos queda, vamos a aplicar un oversampling para finalizar del todo con el muestreo de datos.



Con esta última prueba queda claro que solo sale rentable utilizar la detección de ruido de Tomek-Links

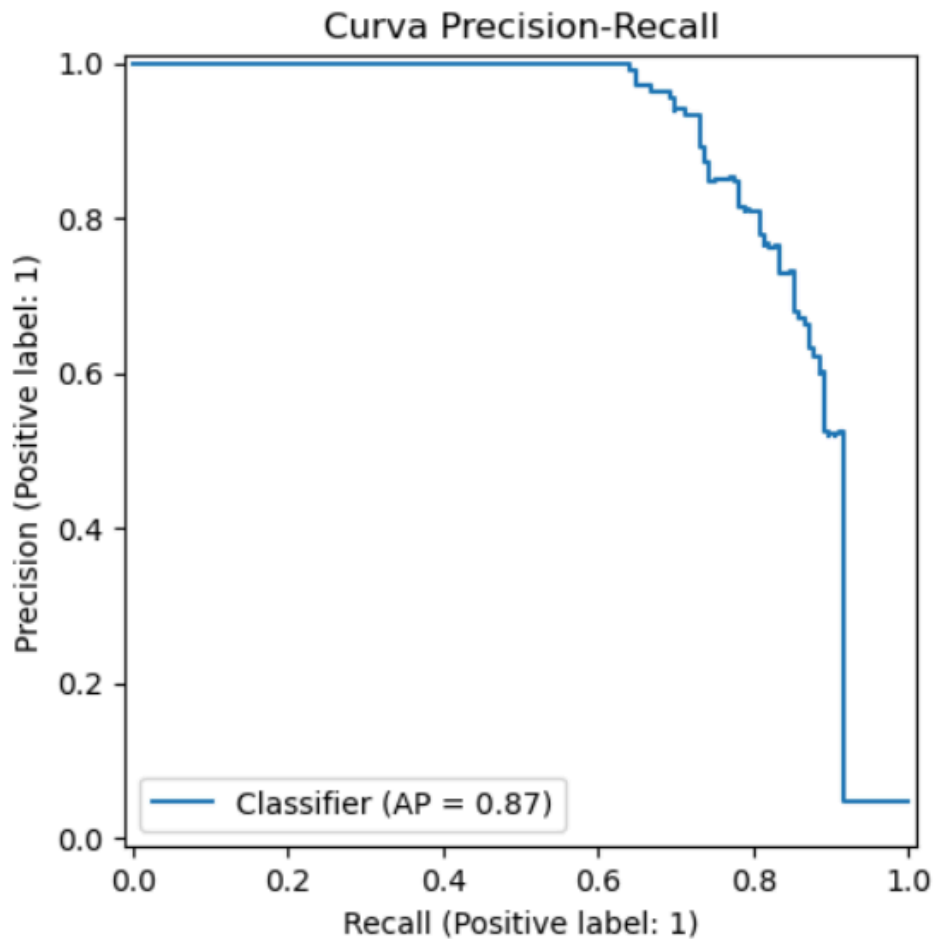
## 7.6. Ajuste de Hiperparámetros y Búsqueda del mejor Umbral

Tras realizar todas las pruebas previstas solo nos quedan los pasos finales. Primeramente tenemos que ajustar los hiperparámetros del KNN, los cuales hasta ahora se habían mantenido por defecto. Para ello usaremos un GridSearchCV, y probaremos los siguientes valores:

- Número de vecinos: 1, 3, 5 y 7
- Pesos: "uniform" y "distance"
- p: 1, 1.5, 2 y 3

Tras correr el GridSearch estos son los resultados obtenidos:

```
Mejor modelo obtenido con los siguientes parametros  
modelo__n_neighbors: 7  
modelo__p: 1.5  
modelo__weights: distance
```



Una vez encontrado el mejor modelo posible, vamos a buscar el mejor de los umbrales, para dar por terminadas las pruebas con el KNN.

```
Mejor umbral=0.738, Fscore en validacion=0.82014
```

## 7.7 Experimentos con Árboles de Decisión:

Antes de descubrir cuál es nuestro modelo definitivo, primero vamos a probar a cambiar el modelo completamente por un árbol de decisión. Como ya hemos comentado, a pesar de que no sea lo óptimo, vamos a mantener el mismo preprocesamiento utilizado para KNN.

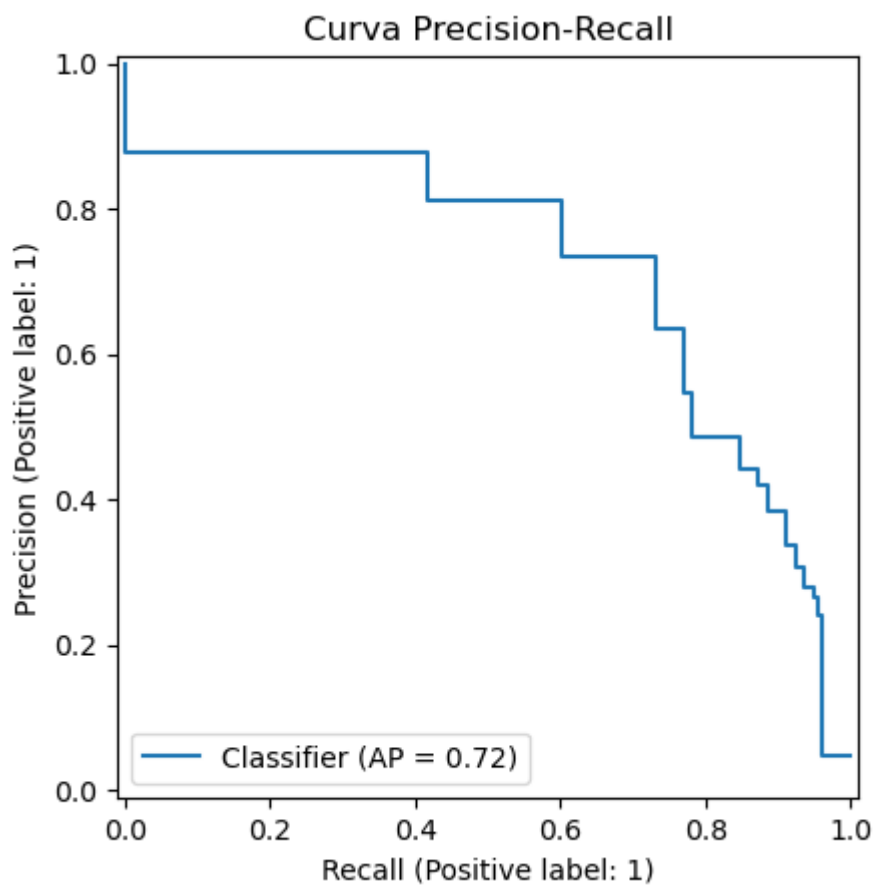
Esto lo hacemos así porque de otra manera deberíamos volver a realizar todas las pruebas con este nuevo modelo y nos llevaría más tiempo del que disponemos.

Al igual que con KNN usaremos un GridSearchCV para ajustar los hiperparámetros. Los valores que vamos a probar serán los siguientes:

- Criterio: Gini, Entropy
- min\_samples\_split: 0.01, 0.05 y 0.1
- min\_samples\_leaf: 0.01, 0.05 y 0.1
- Profundidad máxima: 10, 50, 100
- Alpha: 0, 0.01 y 0.1

Tras correr el GridSearch este es el resultado obtenido:

```
Mejor modelo obtenido con los siguientes parametros
modelo__ccp_alpha: 0.0
modelo__criterion: gini
modelo__max_depth: 50
modelo__min_samples_leaf: 0.01
modelo__min_samples_split: 0.01
```



Ya que estamos buscaremos tambien su mejor umbral y el resultado que este nos dá:

```
Mejor umbral=0.363, Fscore en validacion=0.73312
```

Tras estos resultados resulta evidente que el árbol de decisión no se ajusta lo suficientemente bien al preprocesamiento del KNN, por lo tanto no lo usaremos para nuestro modelo definitivo.

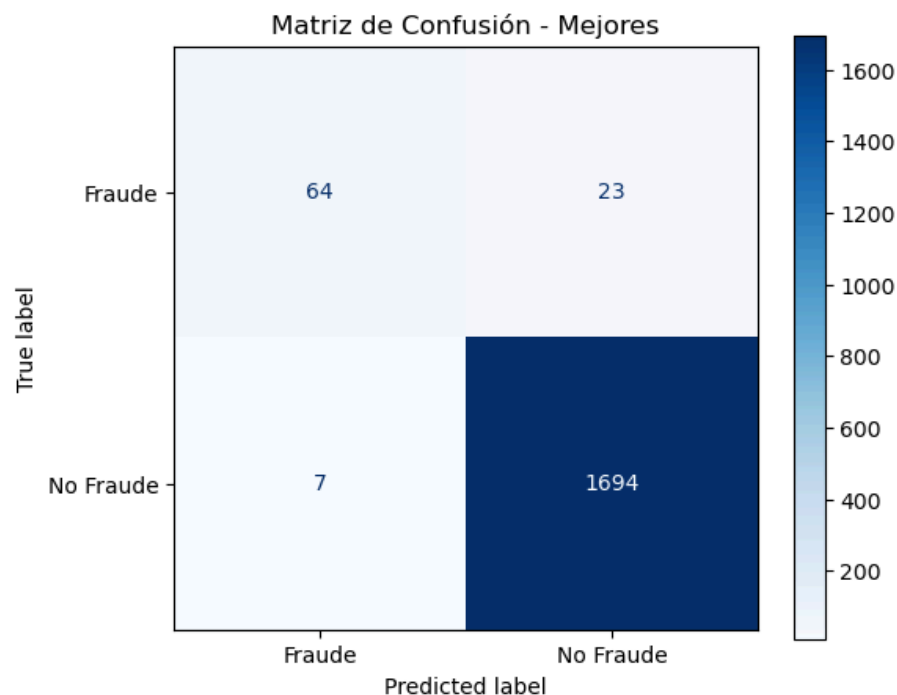
## 8. Modelo Definitivo

Tras todas las pruebas anteriores, hemos llegado a nuestro modelo definitivo. Este modelo cuenta con los siguientes transformadores:

1. Limpiar Espacios
2. Minusculizar
3. Quitar Símbolos
4. Quitar Tildes
5. Rellenar Nulos
  - a. Valores Constantes ("unk" o -1)
  - b. Rellenar Salary Range Discreto
6. Numerizar con Target Encoder y Escalar con StandardScaler
7. Selección de Variables por Correlación con  $k = 9$
8. Tomek Links
9. KNN
  - a. N vecinos = 7
  - b.  $p = 1.5$
  - c. Pesos: distance
  - d. Umbral: 0.738

Tras poner a prueba el modelo con los ejemplos de test obtenemos un rendimiento de 0.8101, muy superior a lo que esperábamos en un principio, dadas las dificultades que implicaba el dataset.

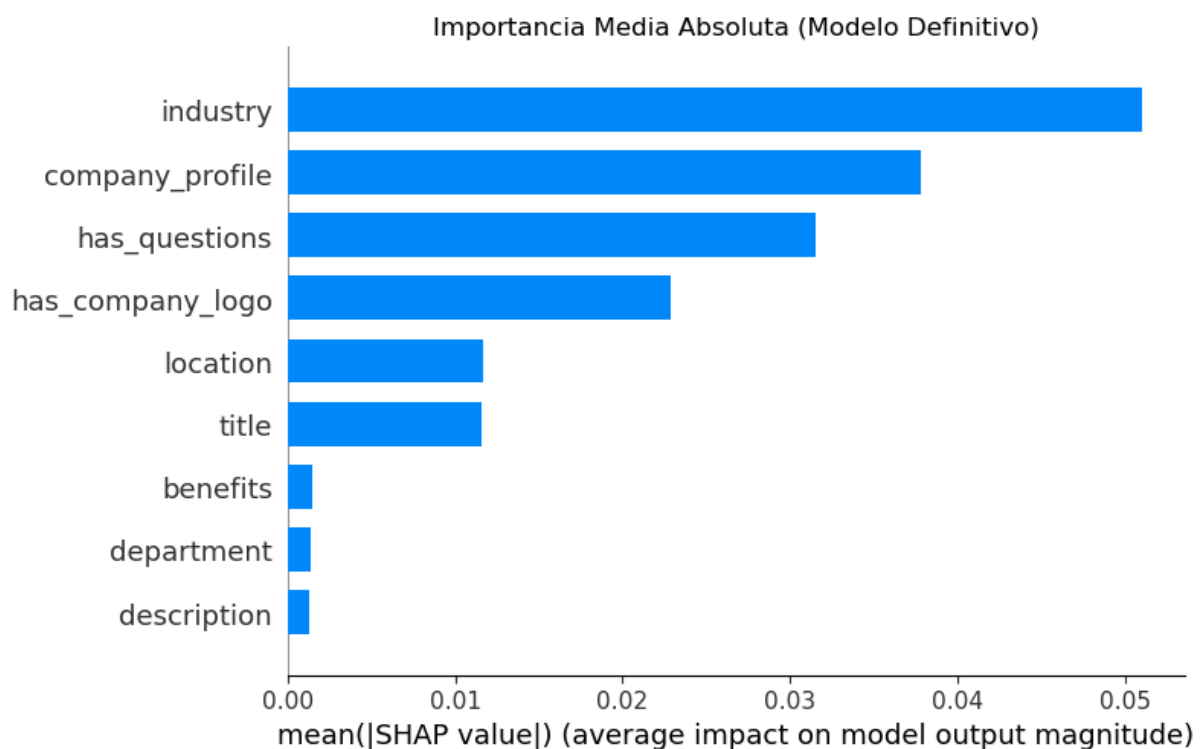
Como último análisis dejamos adjunto la matriz de confusión asociada a este modelo para los datos de test:



# 9. Interpretabilidad

## 9.1. Gráfico de barras

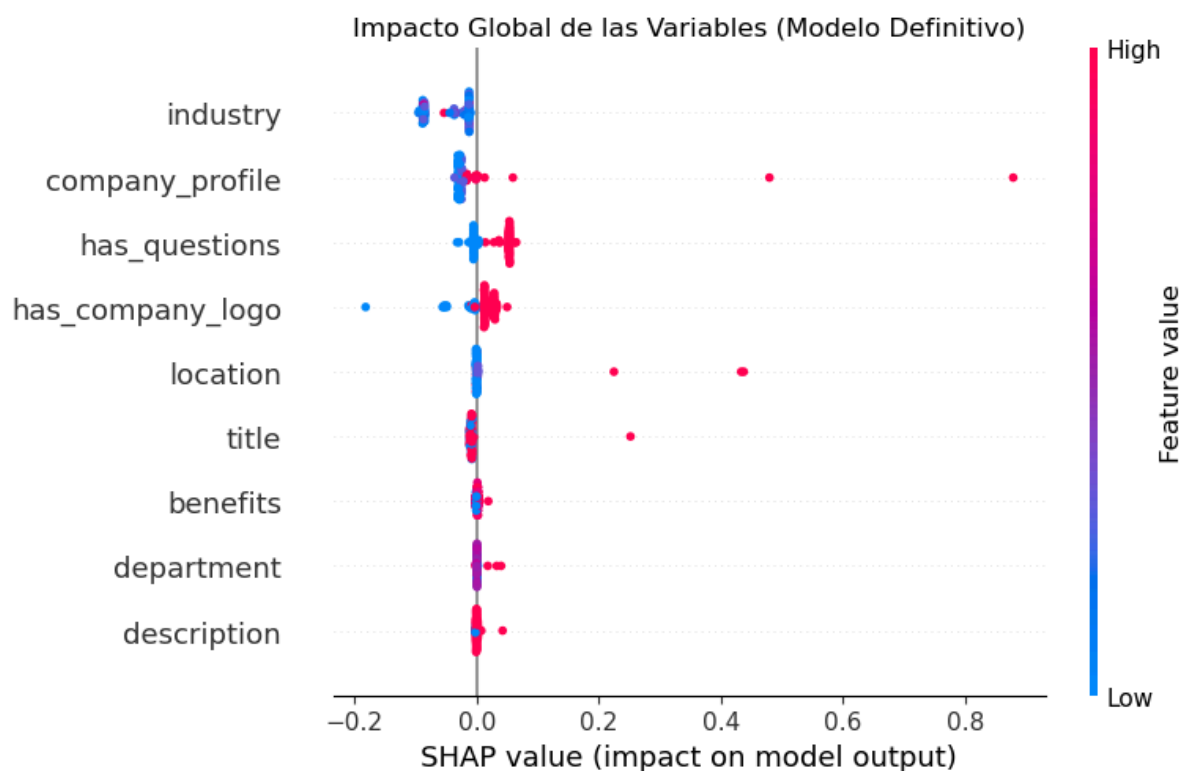
Este gráfico es la representación más directa, este nos indica en qué factores se fija prioritariamente el modelo para tomar una decisión, independientemente de si predice un caso como "Fraude" o "Real". En este sentido, muestra que el modelo depende principalmente de la variable industry, seguida estrechamente por company\_profile) y has\_questions, que son las variables más importantes . Por el contrario, variables como benefits, department y description resultan ser las menos relevantes; apenas tienen un impacto medio en la decisión de este modelo final, tal y como reflejan sus minúsculas barras, lo que demuestra que el algoritmo casi las ignora por completo a la hora de lograr separar las ofertas fraudulentas de las reales.



## 9.2. Gráfico de puntos

El gráfico de puntos ayuda a entender cómo interactúan las variables al aportar tres dimensiones de información simultáneas: la posición en el eje horizontal, donde los puntos a la derecha del cero inclinan la predicción hacia «Fraude» y los de la izquierda hacia «Real»; y el color, donde el rojo representa valores originales «Altos» (categorías sospechosas o respuestas afirmativas tras el escalado) y el azul indica valores «Bajos».

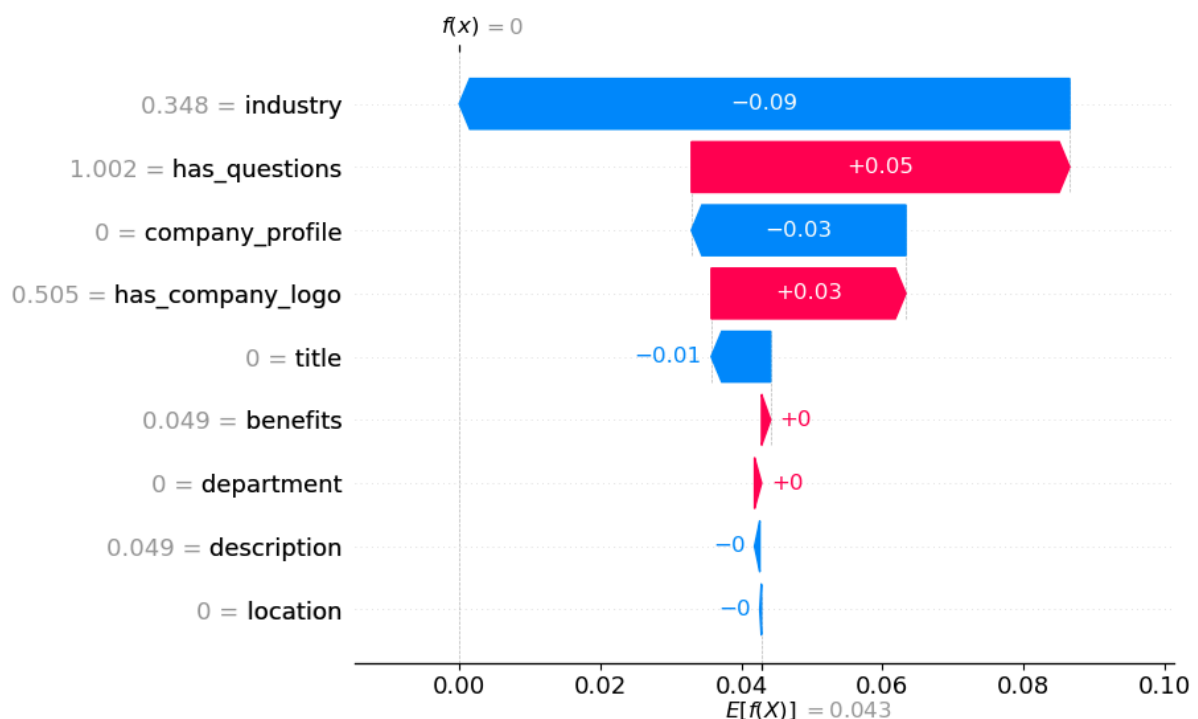
Al observar el comportamiento de los datos, sacamos varias conclusiones clave. Por un lado, la variable `company_profile` presenta puntos rojos muy desplazados hacia la derecha, lo que indica que ciertos perfiles de empresa actúan como alertas y automáticamente la predecimos como fraude. De manera similar, aunque con un impacto más leve, características como `has_questions` y `has_company_logo` muestran que su presencia (puntos rojos) suma cierta sospecha a la oferta, mientras que su ausencia (puntos azules) favorece su legitimidad. Por último, resulta muy interesante el comportamiento de `industry`; a pesar de ser la variable con mayor peso general en el modelo, la gran mayoría de sus observaciones (puntos azules) se agrupan a la izquierda, lo que significa que en la práctica actúa empujando suavemente las decisiones hacia la clasificación de una oferta «Real».



## 9.3. Gráfico de cascada

El Gráfico de Cascada muestra cómo paso a paso el modelo llegó a su decisión final para esa oferta en concreto. Partiendo desde la base, el modelo asume de entrada una probabilidad del 4.3% de que cualquier oferta sea fraudulenta, dado que los datos están desbalanceados y existen muy pocos fraudes en general.

A partir de ese punto inicial, entran en juego las fuerzas a favor del fraude, representadas en color rojo; en este caso concreto, el hecho de que la oferta tuviera preguntas y contar con un logo sumaron, respectivamente, un +0.05 y un +0.03 a la probabilidad de ser falsa. Sin embargo, estas sospechas fueron ampliamente contrarrestadas por las fuerzas a favor de su legitimidad, marcadas en azul. El tipo de industria le restó un enorme -0.09 a la probabilidad total, y la ausencia de ciertas alertas en el perfil de la empresa le restó otro -0.03. En el resultado final, la suma de estas flechas azules de confianza venció claramente a las de sospecha, haciendo que la probabilidad cayera por debajo de cero. En consecuencia, el modelo truncó el valor y predijo un 0% de probabilidad de fraude, concluyendo con total seguridad que se trataba de una oferta 100% real.



# 10. Conclusiones

Tras el análisis y las pruebas realizadas con el dataset de ofertas de empleo, se pueden destacar las siguientes conclusiones:

- Más que el algoritmo utilizado, lo que más mejoró el resultado fue transformar y organizar correctamente la información. La transformación de la variable `salary_range` de un campo de texto con alta ausencia de datos a una variable discreta fue el primer salto en el rendimiento del modelo.
- Como casi todas las variables eran categóricas, convertir datos como la descripción de la empresa en valores útiles para el modelo permitió mejorar notablemente la detección de ofertas falsas.
- Al reducir el ruido y equilibrar mejor las clases, el modelo pudo identificar con más precisión las ofertas fraudulentas sin afectar demasiado a las ofertas reales.
- En este caso, el modelo KNN obtuvo mejores resultados que los árboles de decisión. Sin embargo, esta comparación puede estar influenciada por el hecho de que el proceso de ajuste de hiperparámetros y optimización del pipeline se realizó principalmente sobre KNN, por lo que el preprocesamiento aplicado favorecía especialmente a este modelo.
- Se comprobó que factores como la industria o la información de la empresa son señales importantes para detectar fraude, lo que también ayuda a los usuarios a reconocer ofertas sospechosas.

Lo más destacado del proyecto ha sido el alto rendimiento obtenido frente a la baja calidad inicial de los datos. A pesar de enfrentarnos a un dataset con variables categóricas complejas y numerosos nulos, el modelo demostró un rendimiento inesperado, logrando detectar el fraude con gran precisión. Esto valida que la elección del pipeline de datos fue determinante para superar las carencias de las variables de entrada.